

Adapting BERT for NLP tasks

One pretrained `bert-base-uncased` body, four classical NLP tasks, and three rungs of the adaptation ladder. For every task the delivered model is the one the measurements chose, reported next to the alternatives we trained and rejected.

Team: Jose Angel Pech Xool · Valeria Nicol Hernandez Leon · Lorena Danae Perez Lopez · Julio Cesar de Aquino Castellanos · Gustavo Fuentes Marin · Ricardo Horta Sanchez

Professor: Dexter Gomez · **Course:** Trends in Data Science · **Date:** September 2026

Repository: github.com/joseeangel0/adapting-bert-nlp-tasks

Deliverables: this report, the code that produced every number in it (`scripts/run_experiments.py` reruns the whole grid), and four published models with model cards. Environment: Apple M5 - torch 2.14.0 on mps + Tesla T4 - torch 2.11.0+cu128 on cuda (1 run) · 21 runs · 180 minutes of training.

- | | |
|--------------------------------------|----------------------------|
| 1. What we delivered | 6. Results |
| 2. Task environment | 7. Training diagnostics |
| 3. Hypotheses | 8. Analysis and trade-offs |
| 4. The adaptation ladder, concretely | 9. Publishing |
| 5. Metrics and evaluation | 10. Reproducibility |
| | 11. References |

1. What we delivered

Task	Delivered model	Method	Score	Trainable
Topic classification (AG News)	<code>bert-base-uncased-agnews-topic</code>	Full fine-tuning	93.01 accuracy	109,485,316
Named entity recognition (CoNLL-2003)	<code>bert-base-uncased-conll2003-ner</code>	Full fine-tuning	90.41 entity F1	108,898,569
Part-of-speech tagging (UD English-EWT)	<code>bert-base-uncased-ud-ewt-pos</code>	Full fine-tuning	97.51 token accuracy	108,904,721
Extractive QA (SQuAD v1.1)	<code>bert-base-uncased-squad-qa</code>	Full fine-tuning	82.14 F1	108,893,186

Each of these four models was selected against alternatives trained by us on the same data with the same seed, and every rejected alternative is reported with its numbers in section 6. The short version of what the measurements say:

The gap between a frozen BERT and a fine-tuned one is not a property of BERT — it is a property of the task. It is +3.6 points on part-of-speech tagging, where the information the head needs is already sitting in the pretrained representation, and +57.0 points on extractive QA, where it is not. Choosing the rung of the ladder without measuring is choosing blind.

2. Task environment

All four tasks share one body, `bert-base-uncased` (110 M parameters, 12 encoder layers, hidden size 768). Only the head changes: a sequence classifier over `[CLS]` for topic classification, a per-token classifier for NER

and POS, and a two-logit span head (start, end) for extractive QA.

Task	Dataset	Train	Dev	Test	Labels	License
Topic classification	fancyzhx/ag_news	20,000	5,000	7,600	4	Custom (AG News corpus, academic use)
Named entity recognition	lhoestq/conll2003	14,041	3,250	3,453	9	Reuters corpus terms / research use (CoNLL-2003 shared task)
Part-of-speech tagging	universal-dependencies/universal_dependencies (config en_ewt)	12,544	2,001	2,077	17	CC BY-SA 4.0
Extractive QA	rajpurkar/squad (v1.1)	15,000	3,000	10,570	span	CC BY-SA 4.0

AG News ships no development split, so we carved one out of the training pool with the fixed seed: 20 000 rows for training, 5 000 held out to watch the epochs, and the official 7 600-row test set kept untouched for the numbers in this report. SQuAD is subsampled to 15 000 training examples as the assignment specifies; its test split is not public, so — as is standard for SQuAD v1.1 — the 10 570-question development set is the held-out evaluation set, and a fixed 3 000-question subsample of it is used for the cheaper per-epoch monitoring.

Subword handling

The tokenizer splits words into pieces (playing → play + ##ing) but the datasets carry one label per whole word. We follow the standard convention: **the label goes on the first subword of each word**, and every other position — continuation subwords, [CLS], [SEP], padding — gets -100, the index CrossEntropyLoss ignores. The same first-subword position is used when extracting frozen features, so features and labels stay aligned by construction rather than by luck.

This is the failure mode that does not announce itself: a misalignment either crashes on mismatched lengths or, far worse, trains quietly against shifted labels and produces a healthy-looking loss curve. So before any training ran, scripts/sanity_check.py printed tokens next to their labels and checked the invariants. Its output is committed as docs/sanity_check.txt; an extract:

```
=== ner: subword/label alignment (bert-base-uncased) ===
subword      word_id  label  tag
[CLS]        None    -100   (ignored)
eu           0       3     B-ORG
rejects      1       0     0
german       2       7     B-MISC
...
peter        0       1     B-PER
blackburn    1       2     I-PER

invariant check: {"checked": 500, "length_mismatches": 0,
                  "truncated_sentences": 0, "unexplained_label_loss": 0, "passed": true}
```

The machine check runs over 500 training sentences per token task and asserts that the label array is exactly as long as the input ids and that the number of labelled positions equals the number of words, unless the sentence was genuinely truncated at the maximum length. The same script decodes SQuAD's character-level answer spans back out of the subword start/end indices it produced and compares them to the gold strings, which is the equivalent check for QA: the gold answer is a character offset in the context, and it has to survive being converted into token indices inside an overlapping window that may not even contain it.

lhoestq/conll2003 ships its NER tags as plain integers with no label names attached, so the canonical CoNLL-2003 tag order is pinned in `src/data.py` and verified at load time against the shared task's own example sentence (*EU rejects German call to boycott British lamb .*, with `EU = B-ORG`). A silent change in that mapping would be indistinguishable from a bad model.

Hardware

20 of the 21 runs were trained locally on Apple M5 - torch 2.14.0 on mps — Apple silicon, not a datacentre GPU. Two consequences shaped the design. Mixed precision through `bfloat16` autocast roughly doubles throughput on this device and is enabled for every local run. And MPS has no fused attention kernel that supports dropout, so the models are loaded with the eager attention implementation; on CUDA the same code keeps the fast path. Device selection is automatic (`src/common.get_device`), so the identical modules run on Colab or Kaggle through the notebooks in `notebooks/`.

The remaining run, full fine-tuning on SQuAD, was trained on Tesla T4 - torch 2.11.0+cu128 on cuda through `notebooks/U2T01_colab.ipynb` (section 6.4 explains why). The T4 has no `bfloat16`, so the same code switches to `float16` there. Its score is comparable with the rest; its wall-clock time is not, and every table that shows a time marks it with †. The same notebook re-ran the other twenty runs on the T4 as a cross-hardware replica: none moved by more than 0.3 points (`docs/replica_t4.md`).

3. Hypotheses

We wrote these down before running the grid, from the structure of each task and from section 5.3 of the BERT paper, where feature-based adaptation on CoNLL NER comes within about 0.3 F1 of full fine-tuning when the features are a concatenation of the top four layers — a detail worth holding onto, because we use the last layer alone.

#	Hypothesis	Reasoning	Verdict
H1 · Topic	Feature-based adaptation gets within a few points of fine-tuning.	Topic is a document-level, largely lexical decision; a pooled sentence vector should already separate <i>Sports</i> from <i>Business</i> without reshaping the encoder.	Holds 90.18 vs 93.01: +2.8 points, for zero updated encoder parameters.
H2 · NER	Fine-tuning is required; the frozen gap is large.	Entity boundaries are a contextual, structured decision, and the BIO scheme means a half-recovered span scores zero. A linear read-out of the last layer has no way to reshape the representation for it.	Holds 84.17 vs 90.41 entity F1: +6.2 points, far outside the seed band.
H3 · POS	Frozen features are strong; partial fine-tuning closes most of what is left.	Probing work consistently finds part-of-speech information in the lower and middle layers of BERT. The pretraining objective essentially has to encode it.	Half right Frozen features are strong as predicted (93.94%), but partial fine-tuning closed only half the accuracy gap and almost none of the macro-F1 gap. See 6.3.
H4 · QA	Feature-based adaptation fails outright.	A span head has to make the question interrogate the context. That cross-attention behaviour is not in a frozen masked-language-model representation; no linear layer on top can invent it.	Holds 25.18 vs 82.14 F1: +57.0 points, the widest gap in the assignment. See 6.4 for what the frozen model actually answers.
H5 · Pooling	For frozen sequence classification, mean pooling beats [CLS] .	[CLS] was pretrained for next-sentence prediction, not as a general sentence embedding; without fine-tuning there is nothing that makes it a good summary.	Holds Mean pooling 90.18 vs 88.61 for [CLS] , same classifier. The pooler result in 6.1 is the same effect, larger.

4. The adaptation ladder, concretely

The three rungs are implemented by one function, `src/common.apply_adaptation`, which decides which parameters carry gradients. Everything outside the encoder body is head and always trains.

Rung	What trains	Trainable	How it is done here
Feature-based	Only the consumer of the frozen representation	0 BERT params	Two variants. (a) The last hidden states are extracted once, cached to disk, and consumed by a scikit-learn estimator (logistic regression, linear SVM, random forest). (b) A linear or MLP probe trained with PyTorch on top of the frozen body — identical in spirit, but the result is a normal <code>transformers</code> checkpoint that can be published to the Hub.
Partial fine-tuning	Head + top N encoder layers	≈ 14 M ($N=2$), ≈ 28 M ($N=4$)	The body is frozen, then the top N layers are unfrozen again. The lower layers — which probing work says carry the general lexical and syntactic machinery — are left exactly as pretraining left them.
Full fine-tuning	Everything	≈ 110 M	Every parameter carries a gradient, at the encoder learning rate.

Two learning rates, always

A freshly initialised head and a pretrained encoder want opposite step sizes: the head has no information and needs $\approx 1e-3$ to learn anything in a few epochs, while encoder weights are the product of a long pretraining run and $2e-5$ – $3e-5$ is already enough to damage them. Every run that trains head and body together therefore uses **two parameter groups** (`src/train.TwoGroupTrainer`), with a linear schedule and 10% warmup on both, and weight decay switched off for biases and LayerNorm — the standard exception, because shrinking a normalisation gain toward zero is not regularisation, it is damage. A single shared rate would have crippled every partial and full fine-tuning run, and a crippled run inside a comparison is worse than no run at all.

The feature-based rung deserves one more note. When the body is frozen it is also **pinned in eval mode**, so its dropout never fires and it behaves as a deterministic feature extractor — otherwise the PyTorch probe and the scikit-learn estimator would be consuming different representations of the same sentence, and the comparison between them would mean nothing.

5. Metrics and evaluation

Each task is scored with the metric that penalises the mistakes that matter for it, plus a second metric that catches the specific way that task's models fail quietly.

Task	Primary metric	Why this one	Secondary
Topic classification	Accuracy	AG News is exactly balanced (1 900 test items per class), so accuracy is not misleading here and is the number the literature reports for this dataset.	Macro-F1 — identical to accuracy on a balanced set unless a class has collapsed, which is precisely the failure it is there to expose. A per-class report and a confusion matrix are stored with every run.
NER	Entity-level micro F1 (seqeval)	The unit of correctness is the whole span with its type: predicting B-PER I-PER as B-PER 0 is a wrong answer, not a 50% correct one. This is the official CoNLL-2003 measure, so our numbers sit on the same scale as the published ones.	Token accuracy , kept deliberately: roughly 83% of CoNLL tokens are 0, so a model that predicts nothing at all scores in the eighties. The distance between the two numbers is the point.
POS	Token accuracy	Every token gets exactly one tag, the tags are dense rather than sparse, and there is no span structure to respect — so the per-token decision <i>is</i> the unit of correctness. It is also the standard UD measure.	Macro-F1 over the 17 UPOS tags : accuracy is dominated by NOUN, PUNCT, VERB and DET, and hides whether the rare tags (INTJ, SYM, X) are learned at all. A full 17×17 confusion matrix is stored per run.
Extractive QA	F1 (token overlap, SQuAD official normalisation)	Answers are free-form spans, and a prediction that returns <i>the 1929 stock market crash</i> for a gold <i>1929</i> is largely right. F1 credits partial overlap; both are computed with the official normalisation (lowercase, strip articles and punctuation) and take the maximum over the gold answers.	Exact match — the strict complement. A large EM/F1 spread means the model finds the right region but not the right boundaries, which is a different disease from finding the wrong region.

Where the numbers come from. Every score in this report is computed on a held-out test split that was never used for training or for choosing between methods: the official AG News, CoNLL-2003 and UD English-EWT test splits, and the SQuAD v1.1 development set. The per-epoch curves in section 7 use the development splits.

Training diagnostics. Every run records its full `Trainer` log history (training loss every 50 steps, development metrics every epoch), wall-clock seconds per epoch and per step, total training time, test evaluation time, trainable and total parameter counts, and the resolved hyper-parameters — all serialised to `results/<task>/<run_id>.json`. The figures and tables in this report are generated from those files, so the prose cannot drift away from the numbers.

6. Results

The comparison the assignment asks for — tasks against methods, best run of each rung, primary metric:

Task	Metric	Feature-based	Partial fine-tuning	Full fine-tuning	Delivered
Topic classification (AG News)	Accuracy	90.18	92.36	93.01	Full fine-tuning
Named entity recognition (CoNLL-2003)	Entity F1	84.17	88.10	90.41	Full fine-tuning
Part-of-speech tagging (UD English-EWT)	Token accuracy	93.94	95.66	97.51	Full fine-tuning
Extractive QA (SQuAD v1.1)	F1	25.18	72.91	82.14	Full fine-tuning

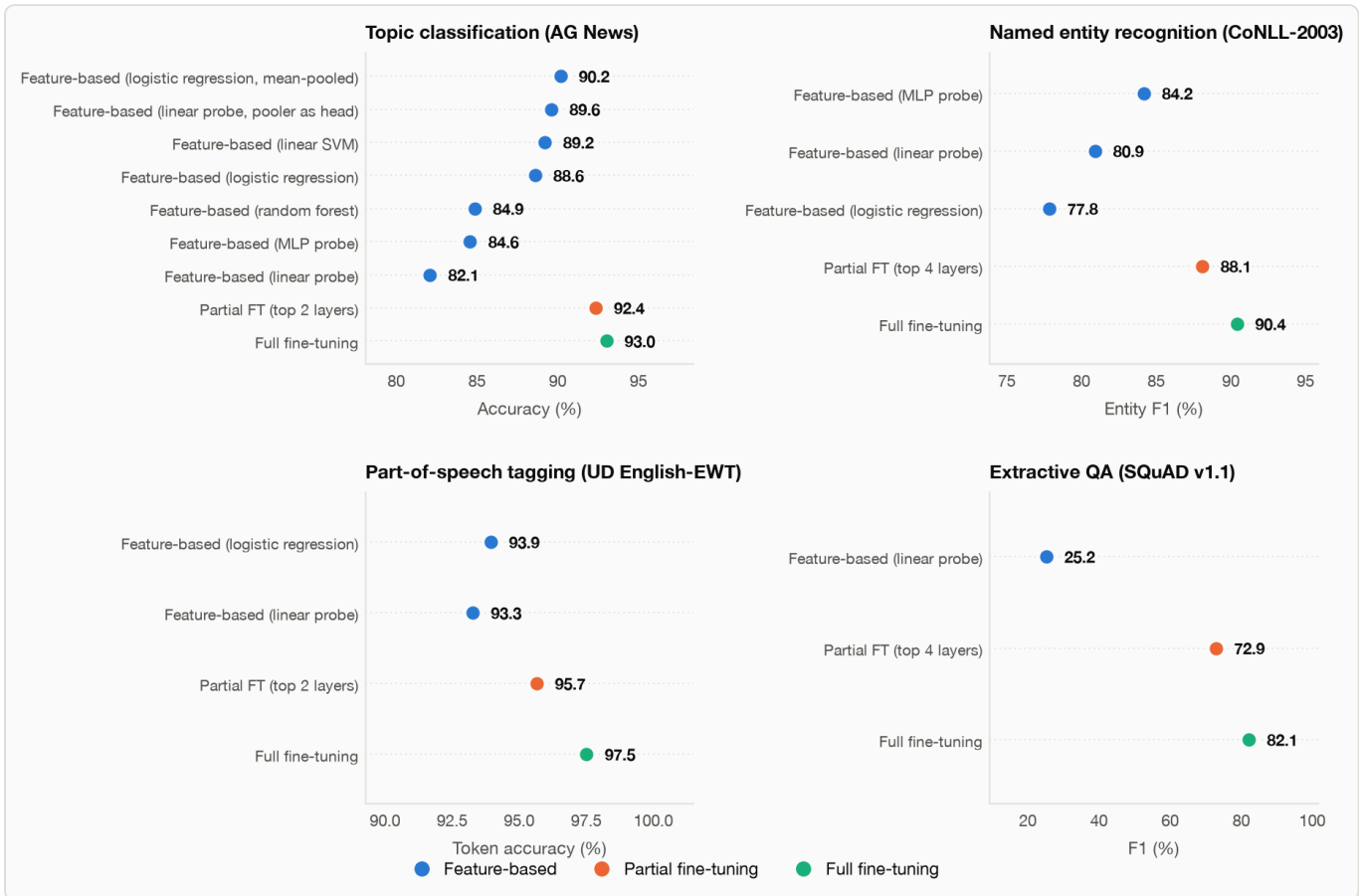


Figure 1 — every run in the grid, on its task's primary metric. Position, not bar length, encodes the score, so the compressed ranges (POS spans 90–97) stay honest without a truncated axis.

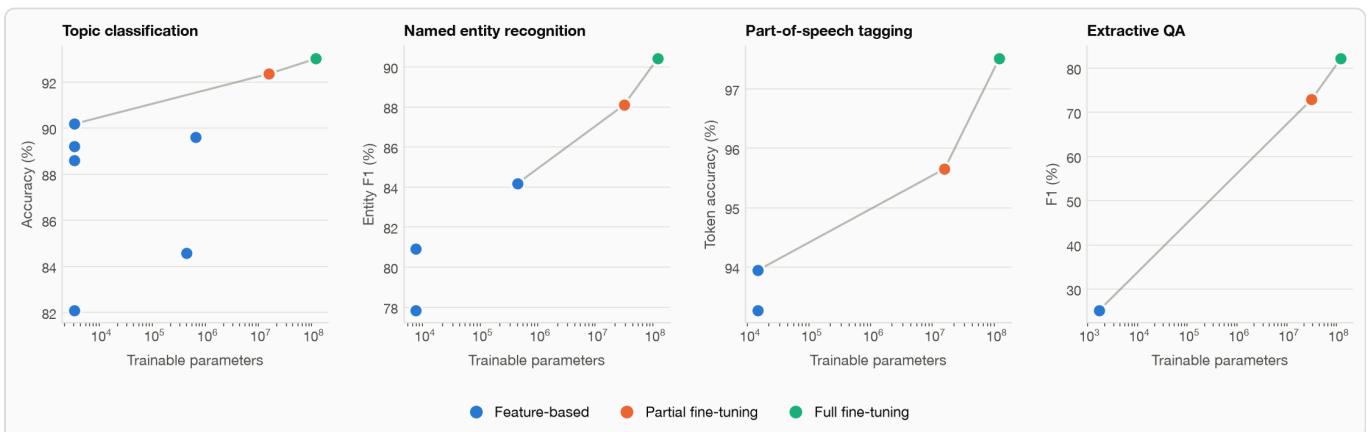


Figure 2 — what each rung costs and what it buys. The horizontal axis is trainable parameters on a log scale; the grey line joins the best run of each rung. A flat line means fine-tuning is not worth paying for. The random forest is absent from this figure alone: its capacity is not a weight count, so it has no honest position on a parameter axis. It appears with every other run in figure 1.

6.1 Topic classification — AG News

Method	Trainable params	Share	Accuracy	Macro F1	Train (min)
Feature-based (logistic regression, mean-pooled)	3,076	0.0%	90.18	90.18	2.9
Feature-based (linear probe, pooler as head)	593,668	0.542%	89.61	89.59	13.6
Feature-based (linear SVM)	3,076	0.0%	89.21	89.20	0.1
Feature-based (logistic regression)	3,076	0.0%	88.61	88.59	2.0
Feature-based (random forest)	0	0.0%	84.87	84.79	0.3
Feature-based (MLP probe)	395,780	0.36%	84.58	84.57	14.8
Feature-based (linear probe)	3,076	0.003%	82.09	82.04	12.5
Partial FT (top 2 layers)	14,178,820	12.95%	92.36	92.35	13.1
Full fine-tuning	109,485,316	100.0%	93.01	93.00	16.6

This is the task where the feature-based rung defends itself. A logistic regression on frozen, mean-pooled last hidden states reaches 90.18% accuracy — +2.8 points below full fine-tuning, for zero updated BERT parameters and 2.9 minutes including feature extraction. **H1 holds.** Topic is a document-level, largely lexical decision, and a pooled sentence vector already separates *Sports* from *Business*; what fine-tuning adds is mostly the *Business* / *Sci-Tech* boundary, which is where the confusion matrices of all our models concentrate their errors.

Two comparisons inside the frozen rung are worth more than the headline number.

Which vector you pool matters more than which classifier you use. Mean pooling beats [CLS] by 1.6 points with the same logistic regression, while swapping logistic regression for a linear SVM on the same [CLS] features moves 0.6 points and a random forest loses 3.7. **H5 holds**, and the reason is architectural: [CLS] was pretrained to carry a next-sentence prediction decision, not to be a sentence embedding. Nothing in the pretraining objective makes it a good summary of the text until fine-tuning teaches it to be one. The random forest result is the expected one — axis-aligned splits are a poor fit for a dense 768-dimensional space where the signal is spread across directions rather than concentrated in individual coordinates.

The pooler is a bottleneck, and it is worth 6.5 points.

Our PyTorch linear probe scored 82.09% while a logistic regression on the same frozen model scored 88.61%. Both are linear models on a frozen BERT, so the gap should have been noise. It is not, and the cause is that `BertForSequenceClassification` does not classify the raw [CLS] state: it classifies `tanh(dense([CLS]))`, BERT's *pooler*, a projection pretrained for next-sentence prediction. Frozen, that tanh-saturated NSP vector is what the probe has to work with. Giving the head more capacity barely helps — an MLP probe on the same frozen pooler reaches only 84.58% — but letting the pooler count as head instead of as frozen body recovers the gap at 89.61%, with still zero encoder parameters updated. The lesson generalises: *frozen* is not a single thing, and where you draw the line between body and head changes the answer more than the choice of classifier does.

Delivered: Full fine-tuning, at 93.01% accuracy. Partial fine-tuning of the top two layers reaches 92.36% with 14.2 M trainable parameters against full fine-tuning's 110 M — 93.01% for eight times the trainable weights. That difference sits inside the ± 1 –3 point seed band the assignment warns about, so we do not claim full fine-tuning is better here; we claim the two are indistinguishable at this sample size and that **partial fine-tuning is the better engineering choice**. We publish the full fine-tuned model only because it is marginally ahead on the measurement we actually have, and we say plainly in its model card that the partial variant is within noise at a fraction of the cost.

6.2 Named entity recognition — CoNLL-2003

Method	Trainable params	Share	Entity F1	Token accuracy	Train (min)
Feature-based (MLP probe)	398,345	0.364%	84.17	97.25	4.2
Feature-based (linear probe)	6,921	0.006%	80.90	96.58	4.3
Feature-based (logistic regression)	6,921	0.0%	77.84	96.10	1.2
Partial FT (top 4 layers)	28,358,409	26.041%	88.10	97.71	4.6
Full fine-tuning	108,898,569	100.0%	90.41	98.08	10.6

H2 holds, and the metric choice is what makes it visible. The frozen probe reaches 84.17 entity F1 against 90.41 for full fine-tuning — a gap of +6.2 points, far outside seed noise. Token accuracy tells a completely different and much more flattering story: the same frozen probe is already above 97% there, because 82.5% of CoNLL test tokens are `0` and a model that finds nothing at all still scores in the eighties. Entity-level F1 is the honest measure, because the unit of correctness is the whole span with its type; recovering `B-PER` but missing the `I-PER` that follows it is a wrong answer, not a half-right one.

Unlike topic classification, this task genuinely needs the encoder reshaped. Deciding that *Washington* is a person here and a location there is a contextual, structured judgement, and a linear read-out of the last layer has no mechanism to make it. Head capacity helps a little — the MLP probe gains 6.3 points over logistic regression on the same frozen features — but it plateaus well below anything that touches the encoder, which is exactly the signature of a representation that lacks the information rather than a head that lacks the power to extract it.

Our frozen number is well below the roughly 0.3 F1 gap the BERT paper reports for feature-based adaptation on this dataset, and the difference is not a contradiction: section 5.3 of the paper concatenates the top four hidden layers as features, while we use the last layer alone. That is a deliberate scope decision — the assignment's ladder is about what trains, not about feature engineering on top of a frozen body — but it does mean our feature-based rung is a floor for the method rather than its ceiling.

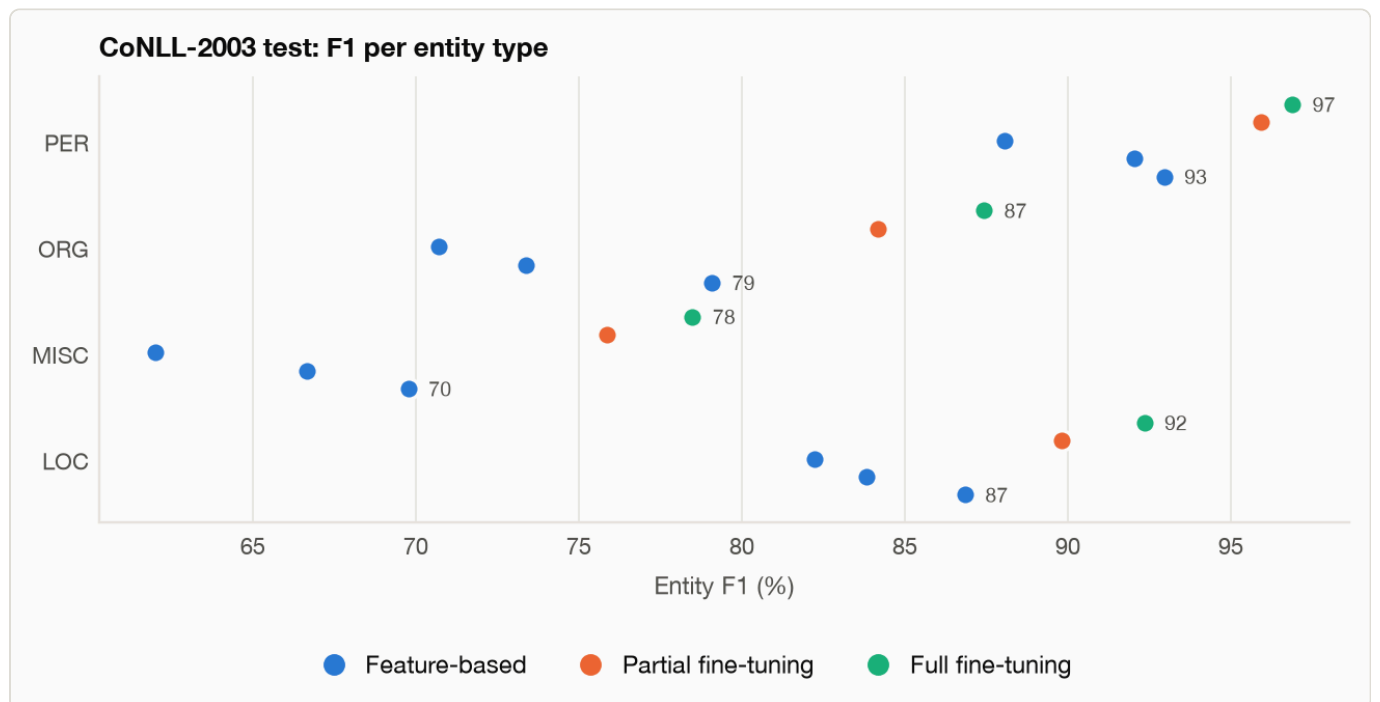


Figure 4 — F1 per entity type. `MISC` is the weakest class for every method, which is expected: it is a residual category holding nationalities, events and adjectival forms with far less internal consistency than `PER`, `ORG` or `LOC`.

Delivered: Full fine-tuning, at 90.41 entity F1. Here the extra cost of fine-tuning buys something that clearly exceeds the noise band, so the top of the ladder is the right answer and the rejected alternatives make the case for it.

6.3 Part-of-speech tagging — UD English-EWT

Method	Trainable params	Share	Token accuracy	Macro F1	Train (min)
Feature-based (logistic regression)	13,073	0.0%	93.94	88.72	1.6
Feature-based (linear probe)	13,073	0.012%	93.27	86.77	4.1
Partial FT (top 2 layers)	14,188,817	13.029%	95.66	89.06	3.6
Full fine-tuning	108,904,721	100.0%	97.51	93.70	11.0

H3 is half right, and the half that is wrong is the interesting one. The first claim holds comfortably: a logistic regression on frozen first-subword states reaches 93.94% token accuracy. Part-of-speech information really is sitting in the pretrained representation, waiting to be read off by a linear model — which is what the layer-wise probing literature predicts, and what a masked-language-model objective essentially has to encode in order to predict a masked token at all.

The second claim — that partial fine-tuning would close most of the remaining gap — does not survive contact with the numbers. Unfreezing the top two layers reaches 95.66%, roughly half of the +3.6-point distance to full fine-tuning's 97.51%. And on macro-F1 it closes almost nothing at all: 88.72 → 89.06, against 93.70 for full fine-tuning. Two metrics that agree about the frozen rung disagree sharply about the middle one, which is the whole reason for reporting both.

Partial fine-tuning made the rarest tag worse than leaving BERT frozen.

On X — 42 test tokens, the residual category for material that fits no other tag — F1 goes 19.8 frozen (logistic regression), 8.2 for the frozen probe, **4.2 for partial fine-tuning**, and 45.2 for full fine-tuning. The same run whose headline accuracy improved by 1.7 points gave up almost everything it had on the rarest class. With only the top two layers free and a head learning rate of 1e-3, the cheapest way to reduce loss is to sharpen the frequent tags; the gradient from 42 tokens cannot compete, and the representation that would have distinguished them is frozen below. Full fine-tuning can move the lower layers and the embeddings, and it is the only configuration that recovers the class. Accuracy alone would have reported this as an unambiguous improvement.

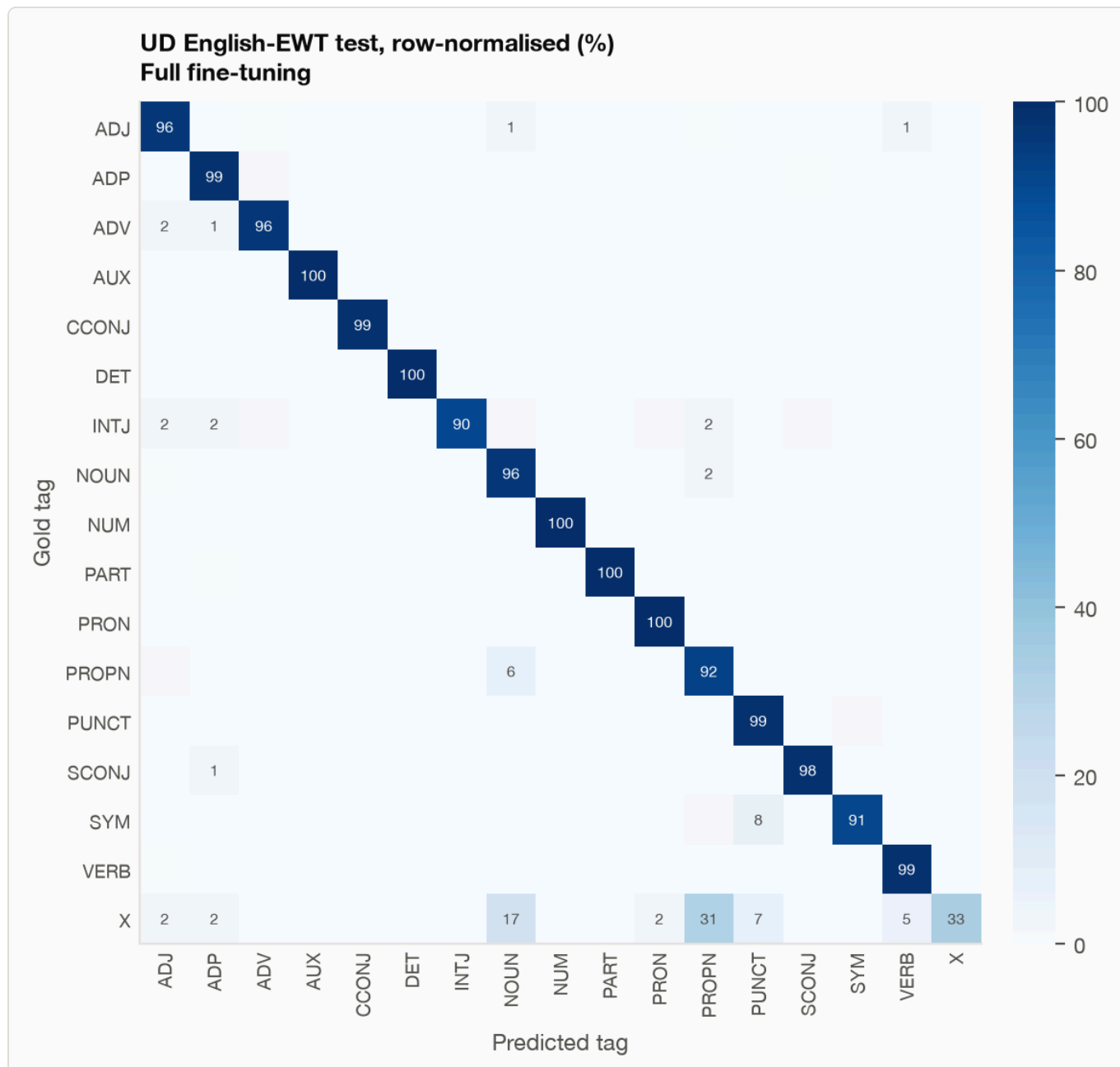


Figure 5 — the delivered POS model on the UD English-EWT test split, row-normalised. The residual errors are the linguistically hard ones rather than random: 6% of `PROPN` is read as `NOUN`, 8% of `SYM` as `PUNCT` — both genuinely ambiguous in web text — and `X` scatters, 31% of it into `PROPN` and 17% into `NOUN`, which is what a residual category with 42 test tokens looks like when it is barely learned.

Delivered: Full fine-tuning, at 97.51% token accuracy and 93.70 macro-F1. This is the one task where we would have been tempted to ship the cheap option on the headline metric — 93.94% for zero updated encoder parameters is a genuinely good tagger — and the secondary metric is what argues against it. If the rare tags matter, and in a tagger feeding a parser they do, the 110 M-parameter model is worth its cost here.

6.4 Extractive question answering — SQuAD v1.1

Method	Trainable params	Share	F1	Exact match	Train (min)
Feature-based (linear probe)	1,538	0.001%	25.18	16.20	25.0
Partial FT (top 4 layers)	28,353,026	26.037%	72.91	60.68	25.7
Full fine-tuning[†]	108,893,186	100.0%	82.14	72.77	8.3

† trained on Tesla T4 - torch 2.11.0+cu128 on cuda instead of Apple M5 - torch 2.14.0 on mps. Its scores compare with the other rows; its wall-clock times do not.

H4 holds, and by the widest margin in the assignment. A span head on a frozen body reaches 25.18 F1; unfreezing the top four encoder layers reaches 72.91, and unfreezing everything 82.14. The full distance, +57.0 points, is an order of magnitude larger than the gap on part-of-speech tagging, on the same body, with the same head learning rate, in the same framework. Whatever "feature-based adaptation" means, it does not mean the same thing from one task to the next.

The reason is visible in the predictions rather than in the score. A frozen BERT plus a linear span head produces answers of the right *kind* and the wrong *identity*; the delivered model, on the same five questions, is in the last column:

Question	Gold answer	Frozen (linear probe)	Delivered (Full fine-tuning)
Which NFL team represented the AFC at Super Bowl 50?	Denver Broncos	Carolina Panthers	Denver Broncos
Which NFL team represented the NFC at Super Bowl 50?	Carolina Panthers	Carolina Panthers	Carolina Panthers
Where did Super Bowl 50 take place?	Santa Clara, California	Super Bowl 50	Santa Clara, California
Which NFL team won Super Bowl 50?	Denver Broncos	Carolina Panthers	Denver Broncos
What color was used to emphasize the 50th anniversary of the Super Bowl?	gold	golden anniversary"	gold

Asked which team represented the AFC, the frozen model answers *Carolina Panthers* — a team name, from the right sentence, and the wrong one, because the NFC team appears too and nothing in a frozen masked-language-model representation makes the token *AFC* in the question reach into the context and discriminate. Asked where the game took place, it returns *Super Bowl 50*: the most salient span in the paragraph, which is what you get when a model has learned what answers look like but not what *this* question wants. The span head is reading a representation that was never asked to let the question interrogate the passage, and a linear layer on top cannot invent that behaviour. The delivered model answers all five correctly. Fine-tuning is not an optimisation here; it is what creates the capability.

The **exact-match / F1 spread** is the second diagnostic worth reading. Partial fine-tuning scores 12.2 points more F1 than EM, and its predictions show why: asked for the AFC team it returns *Denver Broncos defeated the National Football Conference (NFC) champion Carolina Panthers* — the answer is in there, with the boundary in the wrong place. That is a different disease from looking in the wrong place. Full fine-tuning narrows the spread to 9.4 points (72.77 EM against 82.14 F1): the eight lower layers and the embeddings it adds buy tighter boundaries as well as better locations. What is left of the spread is the disease a model trained on 15 000 examples has; more data tightens boundaries, and it is not what separates this model from the frozen one.

Why the full fine-tuning row ran on different hardware. It is the heaviest run in the grid — sequence length 384 with every parameter trainable — and the local attempt did not complete: a machine sleep during it pushed the process into swap, slowing it by two orders of magnitude, and it was stopped rather than left to finish in days. Two attempts on a GPU notebook then failed on arguments that transformers 5 removed from `TrainingArguments`, which the code now tolerates. The third, on a Colab T4 with the unchanged grid, completed, training in 8.3 minutes. Mixing machines inside one comparison needs a check, and the same notebook supplied it: re-run on the T4, partial fine-tuning scored within 0.02 F1 of its local result, and no run in the grid moved by more than 0.3 points (`docs/replica_t4.md`). A 9.2-point gap is not a hardware artefact.

Delivered: Full fine-tuning, at 82.14 F1 and 72.77 exact match, from 108.9 M trainable parameters. Partial fine-tuning of the top four layers is 9.2 points behind with a quarter of the trainable weights — far outside the ± 1 –3 point seed band, so unlike AG News this is not a tie the cheaper option can claim. Two caveats stated plainly. First, this is below published SQuAD numbers for `bert-base` (around 88 F1), which is expected: we train

on the 15 000-example subsample the assignment specifies, roughly a sixth of the corpus. Second, its development F1 was still climbing on the final epoch (+1.4 points), so this run is under-trained rather than converged — the number is a floor for the method, not its ceiling. We report it as measured rather than extending the budget for one task and not the others.

7. Training diagnostics

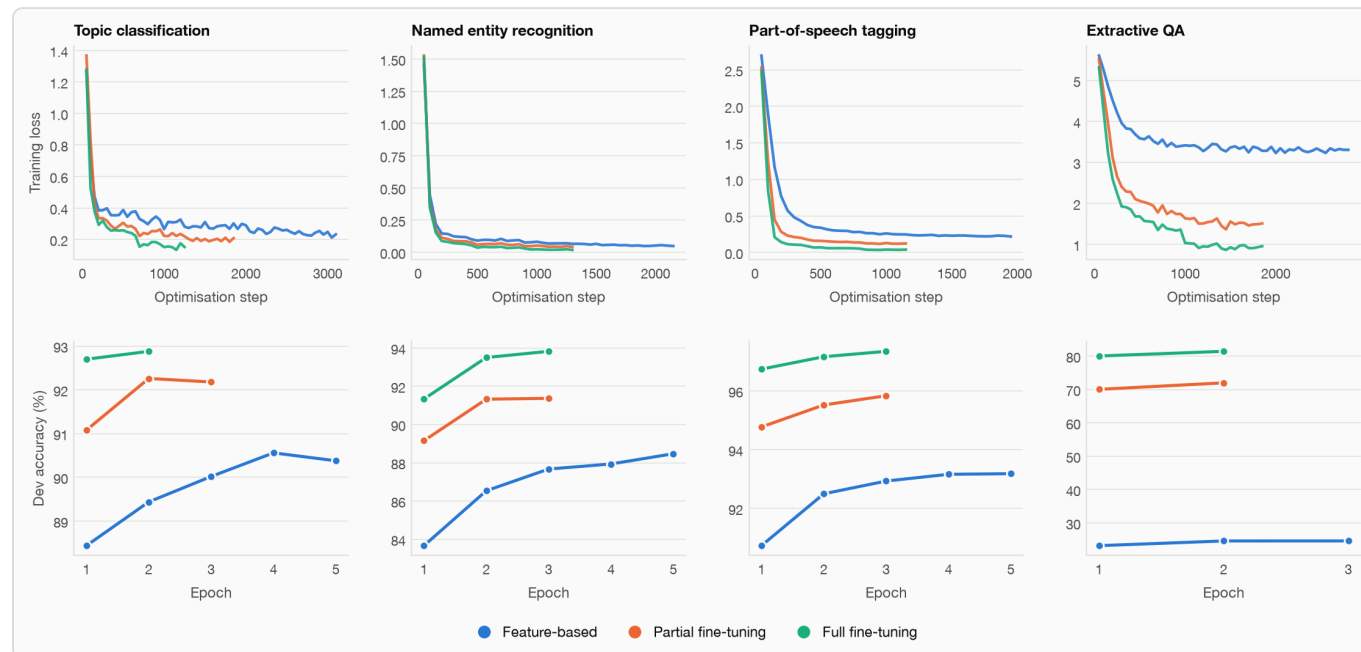


Figure 3 — top: training loss against optimisation step. Bottom: the development metric after each epoch. These are the curves used to decide that runs had converged and that no learning rate was misconfigured.

Three things were being watched in these curves, and all three behaved.

No run was crippled by its learning rate. Every training loss falls smoothly and none diverges, which is the failure a single shared learning rate produces: with $1e-3$ on pretrained weights the loss spikes and never recovers, and with $2e-5$ on a fresh head it crawls. The two-group setup was verified directly as well — across the whole linear schedule the head group keeps a constant multiple of the encoder group's rate, and weight decay stays at zero on biases and LayerNorm gains.

The fine-tuned runs converged; several frozen ones did not. By the last epoch the fine-tuned runs have flattened — partial fine-tuning on AG News moves -0.08 points on its final epoch, which is inside the noise band. The frozen probes are a different story: the AG News MLP probe was still gaining 1.06 points per epoch at epoch five, and both SQuAD fine-tuning runs were still gaining at epoch two (+1.9 F1 partial, +1.4 full). We report these as measured, with the budget fixed in advance for every method, and we flag them here rather than quietly giving the losing configurations extra epochs — which is the cheapest way to manufacture a comparison that says whatever you want. The practical consequence is that **the frozen numbers in this report are floors**: a longer schedule would lift them somewhat, and it would not close a 57-point gap on QA.

Cost scales with what carries a gradient, not with what runs forward. Seconds per optimisation step group cleanly by rung — roughly 0.09–0.19 s frozen, 0.17–0.36 s partial, 0.45–0.74 s full on the token and sequence tasks. A frozen body still does a full forward pass, so the saving is the backward pass and the optimiser update, and it lands around 4–5× rather than the 30 000× the parameter counts might suggest. That gap between *parameters trained* and *time spent* is worth keeping in mind when a parameter count is used as a proxy for cost: it is a good proxy for memory and a poor one for wall clock. The SQuAD full fine-tuning row (†) ran on a T4 GPU, so its times sit on a different clock from every other row and are left out of these ranges.

Task	Method	sec / epoch	sec / step	Total train (min)	Test eval (s)
Topic classification	Feature-based (linear probe, pooler as head)	129	0.206	13.6	49
Topic classification	Feature-based (MLP probe)	140	0.224	14.8	50
Topic classification	Feature-based (linear probe)	117	0.188	12.5	55
Topic classification	Partial FT (top 2 layers)	227	0.363	13.1	44
Topic classification	Full fine-tuning	463	0.741	16.6	47
Named entity recognition	Feature-based (MLP probe)	40	0.091	4.2	10
Named entity recognition	Feature-based (linear probe)	41	0.094	4.3	10
Named entity recognition	Partial FT (top 4 layers)	82	0.186	4.6	10
Named entity recognition	Full fine-tuning	198	0.450	10.6	16
Part-of-speech tagging	Feature-based (linear probe)	43	0.110	4.1	7
Part-of-speech tagging	Partial FT (top 2 layers)	66	0.169	3.6	7
Part-of-speech tagging	Full fine-tuning	213	0.543	11.0	8
Extractive QA	Feature-based (linear probe)	404	0.427	25.0	213
Extractive QA	Partial FT (top 4 layers)	682	0.720	25.7	226
Extractive QA	Full fine-tuning [†]	228	0.241	8.3	65

[†] trained on Tesla T4 - torch 2.11.0+cu128 on cuda instead of Apple M5 - torch 2.14.0 on mps. Its scores compare with the other rows; its wall-clock times do not.

8. Analysis and trade-offs

Four tasks, one body, three rungs. The distance between a frozen BERT and the best thing we could train ranges from +2.8 points to +57.0. That is a factor of about 20, on the same 110 M parameters, and it is the central result of the assignment: **the value of fine-tuning is not a property of the model, it is a property of the task**, and there is no way to read it off a description of either.

8.1 What predicts the gap

The obvious candidate — task difficulty — gets it wrong. Part-of-speech tagging is the easiest task here by any absolute measure (93.94% from a linear model on frozen features) and yet its gap (+3.6) is larger than topic classification's (+2.8). A better predictor is **what kind of operation the head has to perform on the representation**.

- **A read-out of a pooled vector** — topic classification. The information is present and already pooled; the head is a decision boundary in a space that pretraining built. Smallest gap.
- **A read-out per token** — part-of-speech tagging. Same situation, one decision per position. The gap that remains is concentrated in the rare tags, where the frozen representation has the information but too little of the encoder's capacity is allocated to separating them.
- **A contextual, structured judgement** — named entity recognition. Whether *Washington* is a person or a place depends on the sentence, and the answer is scored as a whole span. The representation has to be reshaped, not just read. Medium gap (+6.2).

- **An interaction that pretraining never asked for** — extractive QA. The head needs tokens in the question to interrogate tokens in the context. Masked language modelling never rewarded that behaviour, so no amount of linear read-out recovers it. Gap of +57.0 points, and the frozen model's errors are qualitatively different in kind rather than merely worse in degree.

Stated as a rule: **feature-based adaptation works when the head's job is to read the representation, and fails when its job is to make the representation do something new.** That is a useful prior — but it is a prior, not an answer. Our own hypotheses, written from exactly this reasoning, got H3 half wrong, and the direction of the error (partial fine-tuning damaging the rarest tag) was not something we would have guessed.

8.2 What each rung costs

Trainable parameters are a poor proxy for wall-clock cost. A frozen run trains 0.006% of the weights and still takes about a fifth of the time of a full fine-tune, not a five-thousandth, because the forward pass happens either way; what is saved is the backward pass and the optimiser update. Parameter count is a good proxy for *memory* — optimiser state for 110 M parameters in AdamW is roughly 880 MB that a frozen run simply does not allocate — and it is the reason the heaviest run in this grid needed gradient accumulation to fit alongside a desktop.

Partial fine-tuning is the interesting middle. On AG News it reached 92.36% against 93.01% for full fine-tuning, a difference inside the seed band, for 13% of the trainable weights and 79% of the wall clock. On NER and POS the full model was ahead by more than noise. On POS it was ahead in a way accuracy alone would have hidden. On SQuAD it was ahead by 9.2 points, the widest partial-to-full gap in the grid. There is no single answer, which is the point.

8.3 What we would question in our own results

- **The feature-based rung is a floor, not a ceiling.** We use the last hidden layer alone; section 5.3 of the BERT paper concatenates the top four and comes within 0.3 F1 of full fine-tuning on this very NER dataset, where our gap is +6.2. Our comparison is honest about what it measures — the ladder is about what trains, not about feature engineering — but a reader should not conclude that frozen BERT *cannot* do NER well. It cannot do it well *this way*.
- **Several frozen runs had not converged** when their epoch budget ran out, as section 7 records. Their numbers would rise somewhat with a longer schedule.
- **One seed, one run per configuration**, as the assignment specifies. Every gap below about 3 points in this report — AG News partial against full, the two POS frozen variants — is reported as indistinguishable rather than as a ranking.
- **SQuAD is trained on 15 000 of 87 599 examples**, so its absolute numbers sit well below published `bert-base` results. The comparison between methods is internally valid; the absolute figure is not comparable to the literature.
- **SQuAD's full fine-tuning run was trained on a different machine** from the other twenty (a Colab T4, see 6.4). Its score is comparable — the same notebook re-ran the whole grid on the T4 and no run moved by more than 0.3 points — but its wall-clock time is not, and it is marked † wherever a time is shown.
- **"Frozen" turned out to be ambiguous**, and expensively so. Whether BERT's pooler counts as body or as head moved AG News by 7.5 points — more than the entire distance from feature-based to full fine-tuning on that task. Any comparison of adaptation methods should say where it draws that line.

8.4 What we would do next

The obvious next rung is the one this assignment stops short of: parameter-efficient methods such as LoRA or adapters, which add a small number of trainable parameters *inside* the encoder rather than choosing a prefix of layers to unfreeze. On the evidence here they would be most valuable exactly where partial fine-tuning disappointed — on POS, where freezing the lower layers is what starved the rare tags, and on QA, where the missing capability is an interaction that top-layer tuning has to reconstruct from scratch.

On noise. This is one run per configuration with a fixed seed, as the assignment specifies. Identical code retrained with a different seed moves results by roughly $\pm 1\text{--}3$ points — head initialisation, shuffling order and dropout all contribute — so any gap in this report smaller than that band is noise, not a finding, and we mark it as such rather than re-running it. The gaps we draw conclusions from are the large ones.

9. Publishing

The four delivered models and their tokenizers are published to the HuggingFace Hub by `scripts/push_to_hub.py`, which generates each model card from the same `results/*.json` that produced this report. Every card states the training data and its licence, the adaptation method with its full hyper-parameters, the metrics on the held-out split, **the complete table of methods we tried for that task**, intended use, limitations and bias, and references. The cards are committed in `docs/model_cards/`.

Task	Repository	pipeline_tag	Reported metric
Topic classification	joseeangel/bert-base-uncased-agnews-topic	text-classification	Accuracy 93.01
Named entity recognition	joseeangel/bert-base-uncased-conll2003-ner	token-classification	Entity F1 90.41
Part-of-speech tagging	joseeangel/bert-base-uncased-ud-ewt-pos	token-classification	Token accuracy 97.51
Extractive QA	joseeangel/bert-base-uncased-squad-qa	question-answering	F1 82.14

10. Reproducibility

Seed 42 is fixed for Python, NumPy and PyTorch, and passed to `TrainingArguments` as both `seed` and `data_seed`, so head initialisation and shuffling repeat. The dataset subsamples (AG News 20 000 rows and its 5 000-row dev split, SQuAD 15 000 rows) come from seeded shuffles, so a replica sees the same rows and not merely the same number of rows. Dependencies are pinned in `requirements.txt` at the exact versions used. Reductions on GPU and MPS are not bit-exact, so the last decimal may move — and not much more than that: a teammate re-ran the complete grid on a Colab T4, with a different device, torch build and reduced-precision format, and every run landed within 0.3 points of the numbers in this report (`docs/replica_t4.md`).

```
python3.12 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python scripts/sanity_check.py      # subword/label alignment, before any training
.venv/bin/python scripts/run_experiments.py   # the whole grid, resumable
.venv/bin/python scripts/make_figures.py      # figures from results/*.json
.venv/bin/python scripts/build_report.py      # this report
```

The grid is resumable: a run whose result file already exists is skipped, so an interrupted session continues where it stopped. `--task`, `--limit` and `--force` narrow it while iterating, and `--limit 200` runs the entire pipeline end to end in a couple of minutes as a smoke test.

11. References

- Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL. [arXiv:1810.04805](#) — section 5.3 is the feature-based versus fine-tuning comparison this assignment mirrors.
- Tunstall, L., von Werra, L. & Wolf, T. *Natural Language Processing with Transformers*. O'Reilly, chapters 1–3.
- HuggingFace. *Fine-tune a pretrained model*, and the token-classification and question-answering task guides — the reference recipes for the `-100` convention and for SQuAD windowing and span post-processing, which `src/encoding.py` and `src/qa_utils.py` follow.

- Zhang, X., Zhao, J. & LeCun, Y. (2015). *Character-level Convolutional Networks for Text Classification*. NeurIPS — the AG News corpus.
- Tjong Kim Sang, E. F. & De Meulder, F. (2003). *Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition*. CoNLL.
- Silveira, N. et al. (2014). *A Gold Standard Dependency Corpus for English*. LREC — UD English-EWT, CC BY-SA 4.0.
- Rajpurkar, P., Zhang, J., Lopyrev, K. & Liang, P. (2016). *SQuAD: 100,000+ Questions for Machine Comprehension of Text*. EMNLP.
- Nakayama, H. (2018). *seqeval: A Python framework for sequence labeling evaluation* — the entity-level scorer used for NER.
- Tenney, I., Das, D. & Pavlick, E. (2019). *BERT Rediscovered the Classical NLP Pipeline*. ACL — the layer-wise probing result behind hypothesis H3.